

Dagua Exhaustive Glossary and Reference

Generated automatically

Contents

Generation note. This reference was generated from the codebase, curated glossary metadata, and scripted visuals. Sample visuals use 30 layout steps to keep rebuilds practical.

1 Basics and Terminology

DAG Directed acyclic graph. Edges encode direction, and there are no directed cycles.

Rank A layer index assigned during hierarchical layout. Nodes in the same rank are peers.

Cluster A visual grouping box around related nodes, optionally nested.

Port The attachment location where an edge exits or enters a node.

Routing The post-layout step that turns node positions into edge curves.

Projection A hard corrective step used to enforce constraints like overlap removal or pins.

Coarsening Compression of the graph into a smaller hierarchy level before layout.

Prolongation Expansion of a coarse solution back to a finer graph level.

Refinement Short optimization passes applied after prolongation.

Flex A soft target with a weight that biases layout without forcing a hard lock.

Pin A node position target, usually hard when weight is infinite.

Tier-1 metric A metric cheap enough to compute routinely at any scale.

Tier-2 metric A sampled or costlier metric used when feasible.

Tier-3 metric A hierarchy- or DAG-specific metric that relies on structural metadata.

2 Optimization Stages

Graph Construction Nodes, edges, labels, clusters, and styles are assembled into a `DaguaGraph`.

Node Sizing Text measurement and padding determine node bounding boxes before layout.

Layering Topological depth is converted into ordered ranks consistent with graph direction.

Coarsening Large graphs are compressed into smaller layer-aware hierarchy levels.

Coarse Optimization The smallest hierarchy level is optimized first to establish global structure.

Prolongation + Refinement The coarse layout is expanded and refined level by level.

Edge Routing Node positions are converted into routed edge curves and ports.

Edge Optimization Bezier control points are refined to improve crossings and curvature.

Label Placement Edge labels are placed after routing to avoid collisions where possible.

Rendering Nodes, clusters, edges, and labels are turned into raster/vector output.

3 Public API Functions

Name	Signature and summary
	<code>animate(graph, config: 'Optional[LayoutConfig]' = None, output: 'Optional[str]' = None, animation_config: 'Optional[AnimationConfig]' = None, **kwargs) -> 'AnimationResult'</code>
Render a faithful optimization animation after layout completes.	
<code>border_from_fill</code>	<code>border_from_fill(fill_hex: 'str', darken: 'float' = 0.5) -> 'str'</code>
Derive border color by darkening the fill in HSL lightness space.	
<code>configure</code>	<code>configure(**kwargs: 'Any') -> 'None'</code>
Set global defaults via flat namespace.	
<code>configure_image_ai</code>	<code>configure_image_ai(provider: 'Optional[str]' = None, api_key: 'Optional[str]' = None, api_key_env: 'Optional[str]' = None, model: 'Optional[str]' = None, base_url: 'Optional[str]' = None) -> 'ImageAIConfig'</code>
Set process-wide defaults for image-to-graph/theme AI calls.	
<code>defaults</code>	<code>defaults(**kwargs: 'Any') -> 'Iterator[None]'</code>

Context manager for
scoped default over-
rides.

draw

```
draw(graph: Any, config: Optional[dagua.config.LayoutConfig]  
= None, output: Optional[str] = None, relayout:  
Optional[bool] = None, use_cached: bool = False,  
direction: Optional[str] = None, backend: Optional[str] =  
None, **kwargs: Any) -> Tuple[Any, Any]
```

Layout and render a
graph in one conve-
nience call.

evaluate

```
evaluate(graph: "'DaguaGraph'", positions:  
'torch.Tensor', tier: 'str' = 'auto') -> 'Dict[str,  
float]'
```

Compute layout qual-
ity metrics for a graph
and position tensor.

export_config

```
export_config(path: 'str') -> 'None'
```

Dump current de-
faults to a YAML or
JSON file.

from_dot

```
from_dot(dot_string: 'str', **kwargs) -> "'DaguaGraph'"
```

Build a DaguaGraph
from a DOT string.

from_igraph

```
from_igraph(ig_graph, **kwargs) -> "'DaguaGraph'"
```

Build a Dagua-
Graph from an
igraph.Graph.

from_image

```
from_image(image_path: 'Union[str, Path]', provider:  
'Optional[str]' = None, model: 'Optional[str]' =  
None, *, api_key: 'Optional[str]' = None, api_key_env:  
'Optional[str]' = None, base_url: 'Optional[str]' =  
None, config: 'Optional[ImageAIConfig]' = None) ->  
'DaguaGraph'
```

Construct a Dagua-
Graph from an image
using an LLM.

from_scipy

```
from_scipy(adj_matrix, labels=None, **kwargs) ->  
'DaguaGraph'
```

Build a DaguaGraph
from a scipy sparse
adjacency matrix.

get_default_backend

```
get_default_backend() -> 'str'
```

Return the current
default render back-
end.

get_defaults

```
get_defaults() -> 'Dict[str, Any]'
```

Return the current effective defaults as a flat dict.

`get_image_ai_config` `get_image_ai_config() -> 'ImageAIConfig'`

Return the current process-wide image AI configuration.

`get_theme` `get_theme(name: 'str') -> 'Theme'`

Look up a built-in theme by name. Returns a deep copy.

`graph_code_from_image` `graph_code_from_image(image_path: 'Union[str, Path]', provider: 'Optional[str]' = None, model: 'Optional[str]' = None, *, api_key: 'Optional[str]' = None, api_key_env: 'Optional[str]' = None, base_url: 'Optional[str]' = None, config: 'Optional[ImageAIConfig]' = None, function_name: 'str' = 'build_graph', include_demo_script: 'bool' = False, output_path: 'str' = 'graph.png') -> 'str'`

Return canonical Dagua code reconstructed from an image.

`graph_dict_from_image` `graph_dict_from_image(image_path: 'Union[str, Path]', provider: 'Optional[str]' = None, model: 'Optional[str]' = None, *, api_key: 'Optional[str]' = None, api_key_env: 'Optional[str]' = None, base_url: 'Optional[str]' = None, config: 'Optional[ImageAIConfig]' = None) -> 'Dict[str, Any]'`

Extract a graph JSON dict from an image using a configured AI provider.

`graph_from_json` `graph_from_json(data: 'Union[Dict, str, Path]') -> "'DaguaGraph'"`

Build a DaguaGraph from JSON data.

`graph_from_yaml` `graph_from_yaml(data: 'Union[str, Path]') -> "'DaguaGraph'"`

Build a DaguaGraph from YAML data.

`graph_script_from_dict` `graph_script_from_dict(graph_dict: 'Dict[str, Any]', function_name: 'str' = 'build_graph', output_path: 'str' = 'graph.png') -> 'str'`

Return a ready-to-run Dagua script with layout and export included.

`graph_to_json` `graph_to_json(graph: "'DaguaGraph'") -> 'Dict[str, Any]'`

Serialize a Dagua-
Graph to a JSON-
compatible dict.

graph_to_yaml

```
graph_to_yaml(graph: "'DaguaGraph'", path:  
'Optional[Union[str, Path]]' = None) -> 'str'
```

Serialize a Dagua-
Graph to YAML.

launch_playground

```
launch_playground(*, device: 'str' = 'cpu', max_nodes:  
'int' = 2000, animation_frames: 'int' = 6) -> 'Any'
```

Return an interactive
widget for exploring
layout parameter in-
tuition.

layout

```
layout(graph: 'Any', config: 'Optional[LayoutConfig]' =  
None, trace: 'Any' = None) -> 'torch.Tensor'
```

Compute layout posi-
tions for all nodes.

layout_all

```
layout_all(graph: "'DaguaGraph'", engines:  
'Optional[List[str]]' = None, timeout: 'float' = 300.0)  
-> 'Dict[str, Optional[torch.Tensor]]'
```

Run all available
competitor engines
on a single graph.

layout_similarity

```
layout_similarity(pos_a: 'torch.Tensor', pos_b:  
'torch.Tensor', edge_index: 'Optional[torch.Tensor]' =  
None, k: 'int' = 5) -> 'Dict[str, float]'
```

Compare two layouts
of the same graph up
to rigid transforms.

load

```
load(source: 'Union[Dict, str, Path]') -> "'DaguaGraph'"
```

Load a Dagua-
Graph from a file
(YAML/JSON), dict,
or string.

load_style

```
load_style(path: 'Union[str, Path]') -> 'Dict[str, Any]'
```

Load a style-only file
(YAML/JSON) into
a dict.

make_fill

```
make_fill(base_hex: 'str', bg_hex: 'str' = '#FAFAFA',  
blend: 'float' = 0.25) -> 'str'
```

Blend base color to-
ward background for
a muted fill.

place_edge_labels

```
place_edge_labels(curves: 'List[BezierCurve]', positions:  
'torch.Tensor', node_sizes: 'torch.Tensor', edge_labels:  
'List[Optional[str]]', graph: 'Optional[Any]' = None) ->  
'List[Optional[Tuple[float, float]]]'
```

Compute collision-avoiding positions for edge labels.

```
poster
poster(graph, positions: 'Optional[torch.Tensor]'
= None, config: 'Optional[LayoutConfig]' = None,
output: 'Optional[str]' = None, poster_config:
'Optional[PosterConfig]' = None, **kwargs) ->
'PosterResult'
```

Render a single cinematic still, with LOD support for huge graphs.

```
render
render(graph: 'Any', positions: 'Any' = None,
config: 'Any' = None, output: 'Optional[Union[str,
Path]]' = None, format: 'Optional[str]' = None,
figsize: 'Optional[Tuple[float, float]]' = None,
dpi: 'Optional[int]' = None, show: 'bool' =
False, title: 'Optional[str]' = None, curves:
'Optional[List[BezierCurve]]' = None, label_positions:
'Optional[List[Optional[Tuple[float, float]]]]' = None,
svg_hover_text: 'bool' = True, backend: 'Optional[str]'
= None, fit_to_canvas: 'Union[bool, float]' = False,
bit_equivalent: 'bool' = False) -> 'Tuple[Any, Any]'
```

Render a graph with computed node positions.

```
reset
reset() -> 'None'
```

Restore all defaults to library initial state.

```
route_edges
route_edges(positions: 'torch.Tensor', edge_index:
'torch.Tensor', node_sizes: 'torch.Tensor', direction:
'str' = 'TB', graph: 'Optional[Any]' = None) ->
'List[BezierCurve]'
```

Compute routed edge curves for a laid-out graph.

```
save
save(graph: "'DaguaGraph'", path: 'Union[str, Path]',
format: 'Optional[str]' = None) -> 'None'
```

Save a DaguaGraph to a file (YAML or JSON).

```
save_style
save_style(config: 'Dict[str, Any]', path: 'Union[str,
Path]') -> 'None'
```

Save a style dict to a YAML/JSON file.

```
set_default_backend
set_default_backend(name: 'Optional[str]') -> 'None'
```

Override the global default render backend.

set_device set_device(device: 'str') -> 'None'

Set the global default device ('cpu' or 'cuda').

set_theme set_theme(name: 'str') -> 'None'

Set the global default theme by name.

theme_code_from_image theme_code_from_image(image_path: 'Union[str, Path]', provider: 'Optional[str]' = None, model: 'Optional[str]' = None, *, api_key: 'Optional[str]' = None, api_key_env: 'Optional[str]' = None, base_url: 'Optional[str]' = None, config: 'Optional[ImageAIConfig]' = None, variable_name: 'str' = 'theme') -> 'str'

Return canonical Dagua theme code reconstructed from an image.

theme_dict_from_image theme_dict_from_image(image_path: 'Union[str, Path]', provider: 'Optional[str]' = None, model: 'Optional[str]' = None, *, api_key: 'Optional[str]' = None, api_key_env: 'Optional[str]' = None, base_url: 'Optional[str]' = None, config: 'Optional[ImageAIConfig]' = None) -> 'Dict[str, Any]'

Extract a theme JSON dict from an image using a configured AI provider.

theme_from_image theme_from_image(image_path: 'Union[str, Path]', provider: 'Optional[str]' = None, model: 'Optional[str]' = None, *, api_key: 'Optional[str]' = None, api_key_env: 'Optional[str]' = None, base_url: 'Optional[str]' = None, config: 'Optional[ImageAIConfig]' = None) -> 'Theme'

Extract a visual theme from a graph image using an LLM.

to_igraph to_igraph(graph: "'DaguaGraph'") -> 'Any'

Export a Dagua-Graph to an igraph.Graph.

to_networkx to_networkx(graph: "'DaguaGraph'") -> 'Any'

Export a Dagua-Graph to a NetworkX DiGraph.

to_pyg to_pyg(graph: "'DaguaGraph'") -> 'Any'

Export a Dagua-
Graph to a
torch_geometric.data.Data
object.

to_scipy to_scipy(graph: "'DaguaGraph'") -> 'Any'

Export a Dagua-
Graph to a
scipy.sparse.csr_matrix
adjacency matrix.

tour tour(graph, positions: 'Optional[torch.Tensor]'
= None, config: 'Optional[LayoutConfig]' = None,
output: 'Optional[str]' = None, tour_config:
'Optional[TourConfig]' = None, **kwargs) ->
'AnimationResult'

Render a cinematic
tour of the final or
current graph layout.

4 Graph Construction and Orchestration Methods

Method	Signature and summary
_cluster	add_cluster(self, name: 'str', members: 'Union[List[Any], Dict]', style: 'Optional[ClusterStyle]' = None, label: 'Optional[str]' = None, parent: 'Optional[str]' = None, strict: 'bool' = True) -> 'None'
Add a cluster. Mem- bers can be node IDs/indices or nested dict.	
add_edge	add_edge(self, source: 'Any', target: 'Any', label: 'Optional[str]' = None, type: 'str' = 'normal', style: 'Optional[EdgeStyle]' = None, weight: 'Optional[float]' = None) -> 'None'
Add an edge between two nodes.	
add_node	add_node(self, node_id: 'Any', label: 'Optional[str]' = None, type: 'str' = 'default', style: 'Optional[NodeStyle]' = None) -> 'int'
Add a node and return its integer index.	
align	align(self, node_ids: 'List[Any]', axis: 'str' = 'x', weight: 'float' = 5.0) -> 'None'
Align a group of nodes on an axis.	
cache_layout	cache_layout(self, positions: 'torch.Tensor') -> 'torch.Tensor'

Cache positions as the current fresh layout.

cache_routing cache_routing(self, curves, label_positions:
 'Optional[List[Optional[Tuple[float, float]]]]' = None)
 -> 'None'

Cache routed edges and optional label positions for the current revision.

cluster cluster(self, name: 'str') -> "'ClusterView'"

Return a cluster view by name.

cluster_children cluster_children(self, name: 'str') -> 'List[str]'

Return direct children of a cluster.

cluster_depth cluster_depth(self, name: 'str') -> 'int'

Walk parent chain, return number of hops (0 = root-level cluster).

clusters_for_node clusters_for_node(self, node: 'Any') ->
 "list['ClusterView']"

Return all clusters containing a node.

compute_node_sizes compute_node_sizes(self, font_family: 'str' = '',
 font_size: 'float' = 8.5) -> 'None'

Compute node sizes from labels if not already cached.

configure configure(self, **kwargs: 'Any') -> 'None'

Set graph-scoped style defaults from flat key-word arguments.

edge edge(self, index: 'int') -> "'EdgeView'"

Return an edge view by integer index.

edges_between edges_between(self, source: 'Any', target: 'Any') ->
 "list['EdgeView']"

Return all directed edges from "source" to "target".

edges_for_node edges_for_node(self, node: 'Any') -> "list['EdgeView']"

Return all edges incident to a node.

export_style export_style(self, path: 'str') -> 'None'

Export this graph's style settings to a YAML/JSON file.

get_style_for_cluster get_style_for_cluster(self, name: 'str') ->
 'ClusterStyle'

Get effective style for a cluster.	
<code>get_style_for_edge</code>	<code>get_style_for_edge(self, idx: 'int') -> 'EdgeStyle'</code>
Get effective style for an edge via 5-level cascade.	
<code>get_style_for_node</code>	<code>get_style_for_node(self, idx: 'int') -> 'NodeStyle'</code>
Get effective style for a node via 5-level cascade.	
<code>invalidate_layout</code>	<code>invalidate_layout(self) -> 'None'</code>
Drop cached layout-derived artifacts.	
<code>leaf_cluster_members</code>	<code>leaf_cluster_members(self, name: 'str') -> 'List[int]'</code>
Recursively collect all leaf node indices (own members + children's members).	
<code>node</code>	<code>node(self, id_or_index: 'Any') -> "'NodeView'"</code>
Return a node view by user ID or integer index.	
<code>node_id</code>	<code>node_id(self, index: 'int') -> 'Any'</code>
Return the user-provided ID for a node index.	
<code>pin</code>	<code>pin(self, node_id: 'Any', x: 'Optional[float]' = None, y: 'Optional[float]' = None, weight: 'float' = inf) -> 'None'</code>
Pin a node's position (soft or hard).	
<code>save</code>	<code>save(self, path, format=None)</code>
Save graph to file (YAML/JSON). Format auto-detected from extension.	
<code>set_back_edge_mask</code>	<code>set_back_edge_mask(self, mask: 'torch.Tensor') -> 'None'</code>
Manually override which edges are back edges.	
<code>to</code>	<code>to(self, device: 'str') -> 'DaguaGraph'</code>
Move graph tensors to a device.	
<code>to_igraph</code>	<code>to_igraph(self)</code>
Export to <code>igraph.Graph</code> . See <code>"dagua.io.to_igraph"</code> .	
<code>to_json</code>	<code>to_json(self) -> 'Dict[str, Any]'</code>

Serialize this graph to a JSON-compatible dict.
 to_networkx `to_networkx(self)`
 Export to NetworkX DiGraph. See “dagua.io.to_networkx”.
 to_pyg `to_pyg(self)`
 Export to torch_geometric.data.Data. See “dagua.io.to_pyg”.
 to_scipy `to_scipy(self)`
 Export to scipy.sparse.csr_matrix. See “dagua.io.to_scipy”.
 to_yaml `to_yaml(self, path=None) -> 'str'`
 Serialize to YAML string (or write to file if path given).

5 Configs and Data Structures

5.1 LayoutConfig

All tunable parameters for the layout engine.

Field	Type	Default	Notes
	Union[float, str]	'balanced'	LayoutConfig field
time_budget_s	Optional[float]	None	LayoutConfig field
node_sep	float	70.0	LayoutConfig field
rank_sep	float	240.0	LayoutConfig field
direction	str	'TB'	LayoutConfig field
steps	int	0	LayoutConfig field
lr	float	0.05	LayoutConfig field
device	str	'cpu'	LayoutConfig field
seed	Optional[int]	42	LayoutConfig field
fidelity_dtype	Optional[torch.dtype]	None	LayoutConfig field
multi_start_k	int	1	LayoutConfig field
route_flat_to_stress	bool	True	LayoutConfig field
force_pipeline	Optional[Literal['tree', 'layered_dag', 'force_directed', 'hybrid', 'hybrid_v2', 'planar', 'stress', 'legacy_monolith']]	None	LayoutConfig field
edge_equalize_polish	bool	True	LayoutConfig field
try_planar_first	bool	False	LayoutConfig field
adaptive_spacing	bool	True	LayoutConfig field

verbose	bool	False	LayoutConfig field
w_dag	float	10.0	LayoutConfig field
w_attract	float	2.0	LayoutConfig field
w_attract_x_bias	float	1.0	LayoutConfig field
w_repel	float	0.2	LayoutConfig field
w_overlap	float	5.0	LayoutConfig field
w_cluster	float	1.0	LayoutConfig field
w_cluster_contain	float	2.0	LayoutConfig field
cluster_aware	bool	True	LayoutConfig field
cluster_side_padding_pt	float	8.0	LayoutConfig field
cluster_label_band_pt	float	26.0	LayoutConfig field
cluster_external_clearance	float	10.0	LayoutConfig field
w_align	float	5.0	LayoutConfig field
w_crossing	float	1.8	LayoutConfig field
w_straightness	float	0.5	LayoutConfig field
w_length_variance	float	8.0	LayoutConfig field
w_stress	float	0.0	LayoutConfig field
w_stress_n_pivots	int	50	LayoutConfig field
w_spacing	float	0.3	LayoutConfig field
w_fanout	float	0.3	LayoutConfig field
w_back_edge	float	0.3	LayoutConfig field
exact_repulsion	bool	False	LayoutConfig field
exact_repulsion_threshold	int	500	LayoutConfig field
negative_sample_k	int	128	LayoutConfig field
use_tree_fast_path	bool	False	LayoutConfig field
use_native_median_transpose	bool	True	LayoutConfig field
native_median_passes	int	4	LayoutConfig field
native_transpose_passes	int	8	LayoutConfig field
brandes_koepf_refine	bool	True	LayoutConfig field
layered_rank_row_snap	bool	True	LayoutConfig field
cluster_aware_x_compaction	bool	True	LayoutConfig field
tiny_multiedge_rank_sep_callback	bool	True	LayoutConfig field
tiny_multiedge_rank_sep_min	float	240.0	LayoutConfig field
insert_dummy_nodes	bool	True	LayoutConfig field
decompose_components	bool	True	LayoutConfig field
component_tiling_crossing_block	bool	True	LayoutConfig field
multilevel_threshold	int	20000	LayoutConfig field
multilevel_min_nodes	int	2000	LayoutConfig field
multilevel_coarse_steps	int	100	LayoutConfig field
multilevel_refine_steps	int	25	LayoutConfig field
rvs_threshold	int	100000	LayoutConfig field
rvs_nn_k	int	20	LayoutConfig field
per_loss_backward	str	'auto'	LayoutConfig field
gradient_checkpointing	str	'auto'	LayoutConfig field
hybrid_device	str	'auto'	LayoutConfig field
execution_mode	Literal['auto', 'standard', 'subset_gpu']	'auto'	LayoutConfig field
subset_gpu_threshold	int	10000000	LayoutConfig field

optimizer_fallback	Literal['auto', 'adam', 'sgd']	'auto'	LayoutConfig field
num_workers	int	0	LayoutConfig field
edge_batch_size	int	0	LayoutConfig field
overlap_check_interval	int	0	LayoutConfig field
repel_amortize_interval	int	2	LayoutConfig field
repel_amortize_threshold	int	10000000	LayoutConfig field
fanout_amortize_interval	int	3	LayoutConfig field
fanout_amortize_threshold	int	10000000	LayoutConfig field
edge_random_fraction	float	0.2	LayoutConfig field
relax_steps	int	0	LayoutConfig field
offload_to_disk	bool	True	LayoutConfig field
flex	Optional['LayoutFlex']	None	LayoutConfig field
edge_opt_steps	int	0	LayoutConfig field
edge_opt_lr	float	0.1	LayoutConfig field
edge_routing	str	'differentiable'	LayoutConfig field
edge_routing_auto_skip_threshold	int	5	LayoutConfig field
w_edge_crossing	float	0.5	LayoutConfig field
w_edge_node_crossing	float	10.0	LayoutConfig field
w_edge_angular_res	float	0.0	LayoutConfig field
w_edge_curvature_consistency	float	0.0	LayoutConfig field
w_edge_curvature_penalty	float	1.0	LayoutConfig field
w_edge_cluster_crossing	float	4.0	LayoutConfig field
use_pipeline	bool	False	LayoutConfig field
algorithm	Optional[str]	None	LayoutConfig field
algorithm_params	dict[str, Any]	dict()	LayoutConfig field

5.2 NodeStyle

Visual style for a node.

Field	Type	Default	Notes
	str	'roundrect'	NodeStyle field
fill	str	' '	NodeStyle field
stroke	str	' '	NodeStyle field
stroke_width	float	0.57	NodeStyle field
stroke_width_override_points	Optional[float]	None	NodeStyle field
stroke_dash	str	'solid'	NodeStyle field
stroke_dash_pattern	Optional[Tuple[float, ...]]	None	NodeStyle field
border_opacity	float	1.0	NodeStyle field
font_family	str	' '	NodeStyle field
font_size	float	9.0	NodeStyle field
font_size_override_points	Optional[float]	None	NodeStyle field
font_color	str	'#2D2D2D'	NodeStyle field
text_align	str	'center'	NodeStyle field
text_valign	str	'center'	NodeStyle field
text_rotation	float	0.0	NodeStyle field

text_wrap	str	'none'	NodeStyle field
text_max_width	Optional[float]	None	NodeStyle field
text_transform	str	'none'	NodeStyle field
text_outline	bool	False	NodeStyle field
text_outline_color	str	'#FFFFFF'	NodeStyle field
text_outline_width	float	1.4	NodeStyle field
text_background	str	''	NodeStyle field
text_background_opacity	float	0.85	NodeStyle field
text_background_padding	Tuple[float, float]	(3.0, 2.0)	NodeStyle field
text_background_corner_radius	float	2.0	NodeStyle field
external_label	str	''	NodeStyle field
external_label_position	str	'bottom'	NodeStyle field
external_label_font_size	float	8.0	NodeStyle field
external_label_font_color	str	''	NodeStyle field
external_label_offset	float	4.0	NodeStyle field
padding	Tuple[float, float]	(11.0, 9.0)	NodeStyle field
corner_radius	Union[float, Tuple[float, float, float, float]]	6.0	NodeStyle field
scale_corner_radius	bool	False	NodeStyle field
opacity	float	1.0	NodeStyle field
gradient	str	'none'	NodeStyle field
gradient_color	str	''	NodeStyle field
gradient_angle	float	0.0	NodeStyle field
base_color	str	'#56B4E9'	NodeStyle field
font_weight	str	'regular'	NodeStyle field
font_style	str	'normal'	NodeStyle field
shadow	bool	False	NodeStyle field
shadow_offset	Tuple[float, float]	(1.5, -1.5)	NodeStyle field
shadow_color	str	'#00000020'	NodeStyle field
shadow_blur	float	0.0	NodeStyle field
min_width	Optional[float]	None	NodeStyle field
min_height	Optional[float]	None	NodeStyle field
auto_size_to_label	bool	False	NodeStyle field
aspect_ratio	Optional[float]	None	NodeStyle field
overflow_policy	str	'shrink_text'	NodeStyle field
min_font_size	float	5.0	NodeStyle field
auto_expand_on_floor_overflow	bool	True	NodeStyle field
label_format	str	'plain'	NodeStyle field
border_count	int	1	NodeStyle field
border_position	str	'center'	NodeStyle field
stroke_cap	str	'butt'	NodeStyle field
stroke_join	str	'miter'	NodeStyle field
fill_pattern	str	'solid'	NodeStyle field
fill_pattern_colors	Optional[List[str]]	None	NodeStyle field
fill_pattern_values	Optional[List[float]]	None	NodeStyle field
fill_pattern_angle	float	0.0	NodeStyle field
fill_pattern_hole	float	0.0	NodeStyle field

image	str	''	NodeStyle field
image_fit	str	'contain'	NodeStyle field
image_opacity	float	1.0	NodeStyle field
bevel	bool	False	NodeStyle field
bevel_intensity	float	0.5	NodeStyle field

5.3 EdgeStyle

Visual style for an edge.

Field	Type	Default	Notes
	str	'#6B7280'	EdgeStyle field
width	float	1.4	EdgeStyle field
width_override_points	Optional[float]	None	EdgeStyle field
arrow	str	'normal'	EdgeStyle field
tail_arrow	str	'none'	EdgeStyle field
arrow_fill	str	'filled'	EdgeStyle field
arrow_color	str	''	EdgeStyle field
arrow_length	float	12.0	EdgeStyle field
arrow_width	float	9.0	EdgeStyle field
arrowsize	float	1.0	EdgeStyle field
arrow_scale	Optional[float]	None	EdgeStyle field
arrow_node_fraction	float	0.35	EdgeStyle field
arrow_width_ratio	float	0.85	EdgeStyle field
style	str	'solid'	EdgeStyle field
line_cap	str	'butt'	EdgeStyle field
line_join	str	'miter'	EdgeStyle field
opacity	float	0.75	EdgeStyle field
routing	str	'bezier'	EdgeStyle field
label_font_size	float	7.0	EdgeStyle field
font_size_override_points	Optional[float]	None	EdgeStyle field
label_font_color	str	'#2D2D2D'	EdgeStyle field
label_background	str	'#FAFAFA'	EdgeStyle field
label_background_opacity	float	0.85	EdgeStyle field
label_background_padding	Tuple[float, float]	(3.0, 2.0)	EdgeStyle field
label_background_corner_radius	float	2.0	EdgeStyle field
label_font_family	str	''	EdgeStyle field
label_font_weight	str	'regular'	EdgeStyle field
label_position	float	0.5	EdgeStyle field
label_offset	float	8.0	EdgeStyle field
label_side	str	'auto'	EdgeStyle field
curvature	float	0.4	EdgeStyle field
port_style	str	'distributed'	EdgeStyle field
label_avoidance	bool	True	EdgeStyle field
taper	bool	False	EdgeStyle field
taper_width_start	float	3.0	EdgeStyle field
taper_width_end	float	0.5	EdgeStyle field
head_label	str	''	EdgeStyle field

tail_label	str	''	EdgeStyle field
head_label_offset	float	5.0	EdgeStyle field
tail_label_offset	float	5.0	EdgeStyle field
color_gradient	str	'none'	EdgeStyle field
color_gradient_end	str	''	EdgeStyle field
crossing_style	str	'none'	EdgeStyle field
crossing_size	float	6.0	EdgeStyle field
port_indicator	str	'none'	EdgeStyle field
port_indicator_size	float	5.0	EdgeStyle field
port_indicator_color	str	''	EdgeStyle field

5.4 ClusterStyle

Visual style for a cluster box.

Field	Type	Default	Notes
	str	'#F5F5F0'	ClusterStyle field
stroke	str	'#D4D4D4'	ClusterStyle field
stroke_width	float	0.7	ClusterStyle field
stroke_width_override_points	Optional[float]	None	ClusterStyle field
stroke_dash	str	'solid'	ClusterStyle field
corner_radius	float	8.0	ClusterStyle field
padding	float	38.0	ClusterStyle field
label_position	str	'top-left'	ClusterStyle field
font_size	float	9.5	ClusterStyle field
font_size_override_points	Optional[float]	None	ClusterStyle field
font_weight	str	'bold'	ClusterStyle field
font_color	str	'#4A4A4A'	ClusterStyle field
opacity	float	0.32	ClusterStyle field
fill_opacity	Optional[float]	None	ClusterStyle field
border_opacity	Optional[float]	None	ClusterStyle field
label_background	str	''	ClusterStyle field
label_background_opacity	float	0.85	ClusterStyle field
label_background_padding	Tuple[float, float]	(3.0, 2.0)	ClusterStyle field
label_background_corner_radius	float	2.0	ClusterStyle field
font_family	str	''	ClusterStyle field
label_offset	Tuple[float, float]	(10.0, 12.0)	ClusterStyle field
text_wrap	str	'none'	ClusterStyle field
text_max_width	Optional[float]	None	ClusterStyle field
depth_fill_step	float	0.03	ClusterStyle field
depth_stroke_step	float	0.05	ClusterStyle field
depth_stroke_width_step	float	0.0	ClusterStyle field
depth_opacity_step	float	-0.05	ClusterStyle field
depth_font_size_step	float	-0.5	ClusterStyle field
depth_padding_step	float	-3.0	ClusterStyle field
depth_corner_radius_step	float	0.0	ClusterStyle field
font_size_scaling	Literal['by_height', 'fixed']	'by_height'	ClusterStyle field

member_node_style	Optional[NodeStyle]	None	ClusterStyle field
member_edge_style	Optional[EdgeStyle]	None	ClusterStyle field

5.5 GraphStyle

Graph-level visual settings (all render-only, no layout effect).

Field	Type	Default	Notes
_color	str	'#FAFAFA'	GraphStyle field
margin	float	18.0	GraphStyle field
margin_inches	float	0.11	GraphStyle field
dpi	int	96	GraphStyle field
size_inches	Optional[Tuple[float, float]]	None	GraphStyle field
size_force_fit	bool	False	GraphStyle field
pad_inches	float	0.0	GraphStyle field
ratio	Optional[str]	None	GraphStyle field
density_aware_node_shrink	bool	False	GraphStyle field
title_font_size	float	10.0	GraphStyle field
title_font_weight	str	'regular'	GraphStyle field
title_font_color	str	'#2D2D2D'	GraphStyle field
title_font_family	str	' '	GraphStyle field
edge_label_font_size	float	7.0	GraphStyle field
edge_label_background	str	'#FAFAFA'	GraphStyle field
edge_label_background_opacity	float	0.85	GraphStyle field
node_label_secondary_scale	float	0.85	GraphStyle field
max_figsize	Tuple[float, float]	(30.0, 40.0)	GraphStyle field
min_figsize	Tuple[float, float]	(4.0, 3.0)	GraphStyle field

5.6 AnimationConfig

Animation output and capture controls.

Field	Type	Default	Notes
	int	20	AnimationConfig field
dpi	int	140	AnimationConfig field
figsize	Optional[Tuple[float, float]]	None	AnimationConfig field
format	Optional[str]	None	AnimationConfig field
camera	str	'adaptive'	AnimationConfig field
center_on	Optional[Any]	None	AnimationConfig field
bounds	Optional[Tuple[float, float, float, float]]	None	AnimationConfig field
follow_padding	float	80.0	AnimationConfig field
hold_start_frames	int	6	AnimationConfig field
hold_end_frames	int	12	AnimationConfig field
transition_frames	int	6	AnimationConfig field
show_phase_cards	bool	True	AnimationConfig field

max_layout_frames	int	36	AnimationConfig field
max_edge_frames	int	18	AnimationConfig field
overlay_title	bool	True	AnimationConfig field
output	Optional[str]	None	AnimationConfig field
save_frames_dir	Optional[str]	None	AnimationConfig field
codec	str	'libx264'	AnimationConfig field
bitrate	str	'6M'	AnimationConfig field

5.7 PosterConfig

Single-frame showcase export using the cinematic tour camera logic.

Field	Type	Default	Notes
	int	220	PosterConfig field
figsize	Optional[Tuple[float, float]]	None	PosterConfig field
format	Optional[str]	None	PosterConfig field
output	Optional[str]	None	PosterConfig field
scene	str	'auto'	PosterConfig field
keyframes	Optional[List['CameraKeyframe']]	None	PosterConfig field
keyframe_index	Optional[int]	None	PosterConfig field
show_titles	bool	True	PosterConfig field
follow_padding	float	80.0	PosterConfig field
lod_threshold	int	100000	PosterConfig field
detail_node_limit	int	12000	PosterConfig field
label_node_limit	int	160	PosterConfig field
edge_sample_limit	int	60000	PosterConfig field
density_bins	int	280	PosterConfig field
density_gamma	float	0.52	PosterConfig field

5.8 TourConfig

Camera tour controls for final/current graph showcases.

Field	Type	Default	Notes
	int	24	TourConfig field
dpi	int	160	TourConfig field
figsize	Optional[Tuple[float, float]]	None	TourConfig field
format	Optional[str]	None	TourConfig field
output	Optional[str]	None	TourConfig field
scene	str	'auto'	TourConfig field
keyframes	Optional[List[CameraKeyframe]]	None	TourConfig field
hold_start_frames	int	10	TourConfig field
hold_end_frames	int	14	TourConfig field
show_titles	bool	True	TourConfig field
follow_padding	float	80.0	TourConfig field
codec	str	'libx264'	TourConfig field

bitrate	str	'8M'	TourConfig field
save_frames_dir	Optional[str]	None	TourConfig field
lod_threshold	int	100000	TourConfig field
detail_node_limit	int	8000	TourConfig field
label_node_limit	int	120	TourConfig field
edge_sample_limit	int	40000	TourConfig field
density_bins	int	220	TourConfig field
density_gamma	float	0.55	TourConfig field

5.9 Flex

A soft target value with a weight controlling how strongly the layout

Field	Type	Default	Notes
	float	<required>	Flex field
weight	float	1.0	Flex field

5.10 LayoutFlex

Flex targets for layout-affecting properties.

Field	Type	Default	Notes
_sep	Optional[Flex]	None	LayoutFlex field
rank_sep	Optional[Flex]	None	LayoutFlex field
pins	Optional[Dict[Any, Tuple[Optional[Flex], Optional[Flex]]]]	None	LayoutFlex field
align_x	Optional[List[AlignGroup]]	None	LayoutFlex field
align_y	Optional[List[AlignGroup]]	None	LayoutFlex field

5.11 AlignGroup

A group of nodes that should share the same position on one axis.

Field	Type	Default	Notes
	List[Any]	<required>	AlignGroup field
weight	float	5.0	AlignGroup field

6 Loss Functions

Loss	Signature and summary
_crossing_loss_fallback	<code>_crossing_loss_fallback(pos: 'torch.Tensor', edge_index: 'torch.Tensor', alpha: 'float', max_pairs: 'int') -> 'torch.Tensor'</code>

Original crossing proxy for when layer info is unavailable.

```
_crossing_loss_layered      _crossing_loss_layered(pos: 'torch.Tensor',
                           edge_index: 'torch.Tensor', alpha: 'float',
                           max_pairs: 'int', layer_assignments:
                           'Union[List[int], torch.Tensor]') -> 'torch.Tensor'
```

Adjacent-layer crossing loss with virtual node decomposition (vectorized).

```
_curvature_consistency_loss _curvature_consistency_loss(endpoints: 'torch.Tensor',
                              cp: 'torch.Tensor', t_samples: 'torch.Tensor') ->
                              'torch.Tensor'
```

Penalize variance of curvature across all edges.

```
_curvature_penalty_loss     _curvature_penalty_loss(endpoints: 'torch.Tensor',
                              cp: 'torch.Tensor', t_samples: 'torch.Tensor') ->
                              'torch.Tensor'
```

Penalize total curvature — prefer straighter paths.

```
_edge_cluster_crossing_loss _edge_cluster_crossing_loss(points: 'torch.Tensor',
                              cluster_bboxes: 'torch.Tensor', node_cluster_mask:
                              'torch.Tensor', src_list: 'list', tgt_list: 'list',
                              E: 'int') -> 'torch.Tensor'
```

Penalize edge sample points inside foreign cluster bboxes.

```
_edge_crossing_loss         _edge_crossing_loss(points: 'torch.Tensor', E: 'int')
                              -> 'torch.Tensor'
```

Sigmoid-relaxed edge crossing proxy.

```
_edge_node_crossing_loss    _edge_node_crossing_loss(points: 'torch.Tensor', pos:
                              'torch.Tensor', sizes: 'torch.Tensor', src_list:
                              'list', tgt_list: 'list', E: 'int') -> 'torch.Tensor'
```

Penalty for edge sample points close to unrelated node bboxes.

```
_port_angular_resolution_loss _port_angular_resolution_loss(endpoints:
                              'torch.Tensor', cp: 'torch.Tensor', src_list: 'list',
                              tgt_list: 'list') -> 'torch.Tensor'
```

Penalize small angles between incident edges at each node.

```
_rectilinear_crossing_loss   _rectilinear_crossing_loss(segments: 'torch.Tensor',
                              segment_edges: 'torch.Tensor') -> 'torch.Tensor'
```

Penalize soft intersections between rectilinear sub-segments.

<code>_sampled_curvature_consistency_loss</code>	<code>sampled_curvature_consistency_loss(points: 'torch.Tensor') -> 'torch.Tensor'</code>
Penalize variance in discrete sampled curvature across routes.	
<code>_sampled_curvature_penalty_loss</code>	<code>sampled_curvature_penalty_loss(points: 'torch.Tensor') -> 'torch.Tensor'</code>
Penalize total discrete sampled curvature.	
<code>_spacing_consistency_loss_layerlocal</code>	<code>spacing_consistency_loss_layerlocal(pos: 'torch.Tensor', node_sizes: 'torch.Tensor', layers: 'torch.Tensor', offsets: 'torch.Tensor', num_layers: 'int', target_gap: 'float', device: 'torch.device', layer_index: 'Optional[LayerIndex]' = None) -> 'torch.Tensor'</code>
Layer-local spacing consistency for very large graphs (N $\geq$ 100M).	
<code>alignment_loss</code>	<code>alignment_loss(pos: 'torch.Tensor', align_groups: 'List[Tuple[torch.Tensor, float, int]]') -> 'torch.Tensor'</code>
Penalize positional variance within alignment groups.	
<code>back_edge_compactness_loss</code>	<code>back_edge_compactness_loss(pos: 'torch.Tensor', edge_index: 'torch.Tensor', edge_ctx: 'Optional[EdgeBatchLike]' = None) -> 'torch.Tensor'</code>
Penalize horizontal distance in back-edge pairs (target above source).	
<code>cluster_compactness_loss</code>	<code>cluster_compactness_loss(pos: 'torch.Tensor', clusters: 'dict', device: 'torch.device', cache: 'Optional[_ClusterCache]' = None) -> 'torch.Tensor'</code>
Nodes in same cluster attract their cluster centroid (vectorized).	
<code>cluster_containment_loss</code>	<code>cluster_containment_loss(pos: 'torch.Tensor', node_sizes: 'torch.Tensor', clusters: 'dict', cluster_parents: 'Dict[str, Optional[str]]', padding: 'float' = 18.0, device: 'Optional[torch.device]' = None, cache: 'Optional[_ClusterCache]' = None) -> 'torch.Tensor'</code>
Child cluster bboxes must stay inside parent cluster bboxes (vectorized).	

cluster_separation_loss	cluster_separation_loss(pos: 'torch.Tensor', node_sizes: 'torch.Tensor', clusters: 'dict', padding: 'float' = 10.0, device: 'Optional[torch.device]' = None, cluster_parents: 'Optional[Dict[str, Optional[str]]]' = None, cache: 'Optional[_ClusterCache]' = None) -> 'torch.Tensor'
Sibling cluster bounding boxes repel (vectorized). crossing_loss	crossing_loss(pos: 'torch.Tensor', edge_index: 'torch.Tensor', alpha: 'float' = 5.0, max_pairs: 'int' = 2000, layer_assignments: 'Optional[Union[List[int], torch.Tensor]]' = None) -> 'torch.Tensor'
Differentiable crossing proxy using adjacent-layer sigmoid relaxation. dag_ordering_loss	dag_ordering_loss(pos: 'torch.Tensor', edge_index: 'torch.Tensor', node_sizes: 'torch.Tensor', rank_sep: 'float' = 50.0, edge_ctx: 'Optional[EdgeBatchLike]' = None) -> 'torch.Tensor'
Penalize edges whose targets drift above their sources. edge_attraction_loss	edge_attraction_loss(pos: 'torch.Tensor', edge_index: 'torch.Tensor', x_bias: 'float' = 4.0, edge_ctx: 'Optional[EdgeBatchLike]' = None) -> 'torch.Tensor'
Connected nodes pull to- gether. x-bias encourages vertical edges. edge_length_variance_loss	edge_length_variance_loss(pos: 'torch.Tensor', edge_index: 'torch.Tensor', edge_ctx: 'Optional[EdgeBatchLike]' = None) -> 'torch.Tensor'
Penalize relative (scale- invariant) variation in edge lengths. edge_straightness_loss	edge_straightness_loss(pos: 'torch.Tensor', edge_index: 'torch.Tensor', edge_ctx: 'Optional[EdgeBatchLike]' = None) -> 'torch.Tensor'
Penalize horizontal dis- placement between con- nected nodes. fanout_distribution_loss	fanout_distribution_loss(pos: 'torch.Tensor', edge_index: 'torch.Tensor', degree_threshold: 'int' = 5, edge_ctx: 'Optional[EdgeBatchLike]' = None, step: 'int' = 0, edge_is_sampled: 'bool' = False) -> 'torch.Tensor'

Penalize uneven angular distribution of children for high-degree nodes.

```
flex_spacing_loss      flex_spacing_loss(pos: 'torch.Tensor', node_sizes:
                        'torch.Tensor', layer_index: 'Optional[LayerIndex]',
                        target_sep: 'float', weight: 'float') ->
                        'torch.Tensor'
```

Penalize deviation from flex spacing targets.

```
overlap_avoidance_loss overlap_avoidance_loss(pos: 'torch.Tensor',
                                                node_sizes: 'torch.Tensor', padding: 'float'
                                                = 2.0, layer_index: 'Optional[LayerIndex]' =
                                                None, rvs_threshold: 'int' = 100000, sampled_ctx:
                                                'Optional[SampledNodeLike]' = None, debug_callback:
                                                'Optional[Callable[[str], None]]' = None) ->
                                                'torch.Tensor'
```

Return the overlap-avoidance penalty for the active layout state.

```
position_pin_loss      position_pin_loss(pos: 'torch.Tensor', pin_indices:
                        'torch.Tensor', pin_targets: 'torch.Tensor',
                        pin_weights: 'torch.Tensor', pin_mask:
                        'torch.Tensor') -> 'torch.Tensor'
```

Soft penalty pulling pinned nodes toward targets.

```
repulsion_loss         repulsion_loss(pos: 'torch.Tensor', num_nodes:
                        'int', threshold: 'int' = 2000, sample_k: 'int'
                        = 128, layer_index: 'Optional[LayerIndex]' =
                        None, node_sizes: 'Optional[torch.Tensor]' = None,
                        rvs_threshold: 'int' = 5000, rvs_nn_k: 'int' = 20,
                        sampled_ctx: 'Optional[SampledNodeLike]' = None) ->
                        'torch.Tensor'
```

All nodes repel each other.

```
spacing_consistency_loss spacing_consistency_loss(pos: 'torch.Tensor',
                                                    node_sizes: 'torch.Tensor', layer_index:
                                                    'Optional[LayerIndex]', target_gap: 'float' = 25.0)
                                                    -> 'torch.Tensor'
```

Penalize deviation from target horizontal spacing within layers.

```
warn_legacy_cluster_loss_config warn_legacy_cluster_loss_config(cluster_aware: 'bool',
                                                                w_cluster_containment: 'float') -> 'None'
```

Warn when legacy cluster loss knobs are set under cluster-aware layout.

7 Metric Functions

Metric	Signature and summary
<code>_resolution</code>	<code>angular_resolution(pos: 'torch.Tensor', edge_index: 'torch.Tensor', n_samples: 'int' = 10000, seed: 'int' = 42) -> 'Dict[str, float]'</code>
Minimum angle between incident edges at each node (sampled).	
<code>aspect_ratio</code>	<code>aspect_ratio(pos: 'torch.Tensor', node_sizes: 'Optional[torch.Tensor]' = None) -> 'Dict[str, float]'</code>
Bounding box width / height.	
<code>cluster_separation</code>	<code>cluster_separation(pos: 'torch.Tensor', cluster_ids: 'torch.Tensor') -> 'Dict[str, float]'</code>
Ratio of inter-cluster distance to intra-cluster spread.	
<code>compare</code>	<code>compare(pos_dagua: 'torch.Tensor', pos_reference: 'torch.Tensor') -> 'Dict[str, float]'</code>
Compare two layouts using Procrustes disparity.	
<code>composite</code>	<code>composite(metrics: 'Dict[str, float]') -> 'float'</code>
Compute the directed 100-point composite quality score.	
<code>composite_auto</code>	<code>composite_auto(metrics: 'Dict[str, float]', is_semantically_directed: 'Optional[bool]' = None) -> 'float'</code>
Pick composite or composite_undirected based on direction flag.	
<code>composite_large</code>	<code>composite_large(metrics: 'Dict[str, float]') -> 'float'</code>
Composite score for graphs where only “quick()” metrics are available.	
<code>composite_strict</code>	<code>composite_strict(metrics: 'Dict[str, float]') -> 'float'</code>
Strict variant of “composite()” that refuses silent defaults.	
<code>composite_undirected</code>	<code>composite_undirected(metrics: 'Dict[str, float]') -> 'float'</code>

Composite for undirected graphs: drops direction-sensitive metrics.

`compute_all_metrics` `compute_all_metrics(positions: 'torch.Tensor', edge_index: 'torch.Tensor', node_sizes: 'torch.Tensor', clusters: 'Optional[Dict]' = None, direction: 'str' = 'TB') -> 'Dict[str, float]'`

Legacy API — wraps `quick()` for backward compatibility.

`compute_aspect_ratio` `compute_aspect_ratio(pos: 'torch.Tensor', node_sizes: 'torch.Tensor') -> 'float'`

Legacy: width / height.

`compute_dag_fraction` `compute_dag_fraction(pos: 'torch.Tensor', edge_index: 'torch.Tensor', direction: 'str' = 'TB') -> 'float'`

Legacy: fraction of edges in correct DAG direction.

`compute_edge_length_variance` `compute_edge_length_variance(pos: 'torch.Tensor', edge_index: 'torch.Tensor') -> 'float'`

Legacy: variance of edge lengths.

`compute_edge_straightness` `compute_edge_straightness(pos: 'torch.Tensor', edge_index: 'torch.Tensor', direction: 'str' = 'TB') -> 'float'`

Legacy: mean angular deviation from primary axis (degrees).

`compute_mean_edge_length` `compute_mean_edge_length(pos: 'torch.Tensor', edge_index: 'torch.Tensor') -> 'float'`

Legacy: average Euclidean edge length.

`compute_min_node_gap` `compute_min_node_gap(pos: 'torch.Tensor', node_sizes: 'torch.Tensor') -> 'float'`

Legacy: minimum gap between non-overlapping node bboxes (sampled).

`compute_total_area` `compute_total_area(pos: 'torch.Tensor', node_sizes: 'torch.Tensor') -> 'float'`

Legacy: bounding box area.

`compute_x_alignment` `compute_x_alignment(pos: 'torch.Tensor', edge_index: 'torch.Tensor', direction: 'str' = 'TB') -> 'float'`

Legacy: mean cross-axis displacement between connected nodes.

count_crossings	count_crossings(pos: 'torch.Tensor', edge_index: 'torch.Tensor', *, seed: 'Optional[int]' = None) -> 'int'
Count edge crossings exactly on small graphs and by sampling on large ones.	
count_overlaps	count_overlaps(pos: 'torch.Tensor', node_sizes: 'torch.Tensor') -> 'int'
Legacy: count overlapping node pairs (returns int).	
count_overlaps_detailed	count_overlaps_detailed(pos: 'torch.Tensor', node_sizes: 'torch.Tensor', *, seed: 'Optional[int]' = None) -> 'Dict[str, int]'
Count overlapping node bounding box pairs using spatial hashing.	
dag_consistency	dag_consistency(pos: 'torch.Tensor', edge_index: 'torch.Tensor', direction: 'str' = 'TB', back_edge_mask: 'Optional[torch.Tensor]' = None) -> 'Dict[str, float]'
Fraction of edges pointing in the correct DAG direction + violation details.	
depth_position_correlation	depth_position_correlation(pos: 'torch.Tensor', topo_depth: 'torch.Tensor') -> 'Dict[str, float]'
Spearman rank correlation between topological depth and y-coordinate.	
edge_curvature_consistency	edge_curvature_consistency(curves) -> 'Dict[str, float]'
Compute curvature statistics across all edges.	
edge_direction_straightness	edge_direction_straightness(pos: 'torch.Tensor', edge_index: 'torch.Tensor', direction: 'str' = 'TB') -> 'Dict[str, float]'
Mean angular deviation from primary axis (degrees).	
edge_length_cv	edge_length_cv(pos: 'torch.Tensor', edge_index: 'torch.Tensor') -> 'Dict[str, float]'
Edge length coefficient of variation + distribution stats.	
edge_node_crossing_count	edge_node_crossing_count(curves, positions: 'torch.Tensor', node_sizes: 'torch.Tensor', edge_index: 'torch.Tensor', T: 'int' = 10) -> 'Dict[str, Any]'

Count edge-node bbox crossings by sampling T points per bezier.

evaluate

```
evaluate(graph: "'DaguaGraph'", positions:
'torch.Tensor', tier: 'str' = 'auto') -> 'Dict[str,
float]'
```

Compute layout quality metrics for a graph and position tensor.

full

```
full(pos: 'torch.Tensor', edge_index: 'torch.Tensor',
topo_depth: 'Optional[Any]' = None, node_sizes:
'Optional[torch.Tensor]' = None, cluster_ids:
'Optional[torch.Tensor]' = None, direction: 'str' =
'TB', stress_sources: 'int' = 200, stress_targets:
'int' = 1000, crossing_samples: 'int' = 1000000,
neighborhood_samples: 'int' = 5000, curves:
'Optional[Any]' = None, label_positions: 'Optional[Any]'
= None, edge_labels: 'Optional[Any]' = None,
back_edge_mask: 'Optional[torch.Tensor]' = None) ->
'Dict[str, Any]'
```

All metrics including sampled Tier-2 and DAG-specific Tier-3.

graphviz_delta

```
graphviz_delta(dagua_metrics: 'Dict', graphviz_metrics:
'Dict') -> 'Dict[str, float]'
```

How much worse/better is Dagua vs Graphviz for each metric.

label_overlap_count

```
label_overlap_count(label_positions:
'List[Optional[Tuple[float, float]]]', edge_labels:
'list', positions: 'torch.Tensor', node_sizes:
'torch.Tensor', label_font_size: 'float' = 7.0) ->
'Dict[str, int]'
```

Count overlaps between edge labels and node bboxes, and between labels.

layer_uniformity

```
layer_uniformity(pos: 'torch.Tensor', topo_depth) ->
'Dict[str, float]'
```

Uniformity of vertical spacing between adjacent topological layers.

layout_similarity

```
layout_similarity(pos_a: 'torch.Tensor', pos_b:
'torch.Tensor', edge_index: 'Optional[torch.Tensor]' =
None, k: 'int' = 5) -> 'Dict[str, float]'
```

Compare two layouts of the same graph up to rigid transforms.

neighborhood_preservation	neighborhood_preservation(pos: 'torch.Tensor', edge_index: 'torch.Tensor', num_nodes: 'int', n_samples: 'int' = 5000, k: 'int' = 10, all_pairs_dist: 'Optional[np.ndarray]' = None) -> 'Dict[str, float]'
Fraction of k-nearest graph neighbors that are also k-nearest Eu- clidean neighbors.	
overall_quality	overall_quality(metrics: 'Dict[str, float]') -> 'float'
Legacy single scalar summarizing layout quality. Higher is better.	
port_angular_resolution	port_angular_resolution(curves, edge_index: 'torch.Tensor') -> 'Dict[str, float]'
Minimum angle be- tween incident edge tangents at actual ports.	
quick	quick(pos: 'torch.Tensor', edge_index: 'torch.Tensor', topo_depth=None, node_sizes: 'Optional[torch.Tensor]' = None, direction: 'str' = 'TB', back_edge_mask: 'Optional[torch.Tensor]' = None, *, seed: 'Optional[int]' = None) -> 'Dict[str, Any]'
Compute the Tier-1 metric bundle.	
sampled_crossing_rate	sampled_crossing_rate(pos: 'torch.Tensor', edge_index: 'torch.Tensor', n_samples: 'int' = 1000000, *, seed: 'Optional[int]' = None) -> 'Dict[str, float]'
Estimate crossing rate via random edge-pair sampling.	
sampled_stress	sampled_stress(pos: 'torch.Tensor', edge_index: 'torch.Tensor', num_nodes: 'int', n_sources: 'int' = 200, n_targets: 'int' = 1000, max_dist: 'int' = 20, all_pairs_dist: 'Optional[np.ndarray]' = None) -> 'Dict[str, float]'
Estimate graph- theoretic stress from deterministic source samples.	
segments_intersect	segments_intersect(p1: 'torch.Tensor', p2: 'torch.Tensor', p3: 'torch.Tensor', p4: 'torch.Tensor') -> 'torch.Tensor'
Vectorized segment in- tersection test. All in- puts "[N, 2]"	

`within_layer_compactness` `within_layer_compactness(pos: 'torch.Tensor', topo_depth)`
 -> 'Dict[str, float]'

Fraction of each layer's
 x-range that is occupied
 by nodes.

7.1 Metric Keys from `quick()`

`_compute_time_seconds`, `_graph_n_edges`, `_graph_n_nodes`, `aspect_ratio`, `back_edge_count`, `back_edge_fr`,
`bbox_height`, `bbox_width`, `dag_consistency`, `dag_max_violation_mag`, `dag_mean_violation_mag`,
`dag_num_violations`, `depth_spearman_pval`, `depth_spearman_rho`, `edge_length_cv`, `edge_length_max`,
`edge_length_mean`, `edge_length_min`, `edge_length_p01`, `edge_length_p99`, `edge_length_std`,
`edge_straightness_below_15`, `edge_straightness_mean_deg`, `overlap_count`

7.2 Metric Keys from `full()`

`_compute_time_seconds`, `_graph_n_edges`, `_graph_n_nodes`, `_sample_sizes`, `angular_res_below_10deg`,
`angular_res_mean_deg`, `angular_res_median_deg`, `aspect_ratio`, `back_edge_count`, `back_edge_fraction`,
`bbox_height`, `bbox_width`, `composite_score`, `crossing_eligible_pairs`, `crossing_estimated_total`,
`crossing_n_samples`, `crossing_rate`, `crossing_se`, `dag_consistency`, `dag_max_violation_mag`,
`dag_mean_violation_mag`, `dag_num_violations`, `depth_spearman_pval`, `depth_spearman_rho`, `edge_length_cv`,
`edge_length_max`, `edge_length_mean`, `edge_length_min`, `edge_length_p01`, `edge_length_p99`,
`edge_length_std`, `edge_straightness_below_15`, `edge_straightness_mean_deg`, `layer_compactness_mean`,
`layer_compactness_min`, `layer_spacing_cv`, `layer_spacing_max`, `layer_spacing_mean`, `layer_spacing_min`,
`n_layers`, `neighborhood_mean`, `neighborhood_median`, `neighborhood_std`, `overlap_count`, `sampled_stress`,
`stress_n_pairs`, `stress_n_sources`, `stress_n_targets`

8 Optimization Hyperparameters

Name	Display name	Category	Description
<code>_dag</code>	DAG Ordering Strength	forces	How strongly edges are forced to point downward.
<code>w_attract</code>	Edge Attraction	forces	How strongly connected nodes pull toward each other.
<code>w_attract_x_bias</code>	Vertical Edge Preference	forces	Extra weight on horizontal attraction (makes edges more vertical).
<code>w_repel</code>	Node Repulsion	forces	How strongly all nodes push apart from each other.
<code>w_overlap</code>	Overlap Avoidance	forces	Penalty for node bounding box intersection.
<code>w_crossing</code>	Crossing Minimization	aesthetics	Penalty for edge crossings (differentiable proxy).
<code>w_straightness</code>	Edge Straightness	aesthetics	Penalizes horizontal displacement between connected nodes.
<code>w_length_variance</code>	Edge Length Uniformity	aesthetics	Penalizes variance in edge lengths (prefer uniform over minimum).

node_sep	Node Separation	spacing	Minimum horizontal gap between nodes (pixels).
rank_sep	Rank Separation	spacing	Minimum vertical gap between layers (pixels).
steps	Optimization Steps	optimization	Number of gradient descent steps.
lr	Learning Rate	optimization	Step size for gradient descent.
w_cluster_contain	Cluster Containment	forces	How strongly child clusters are kept inside parent clusters.

9 Visual Guide

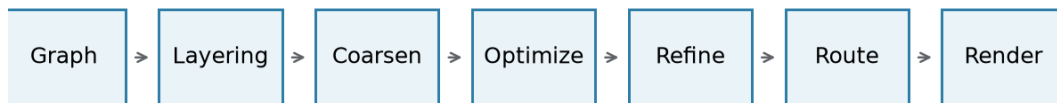


Figure 1: Pipeline schematic from graph construction through rendering.

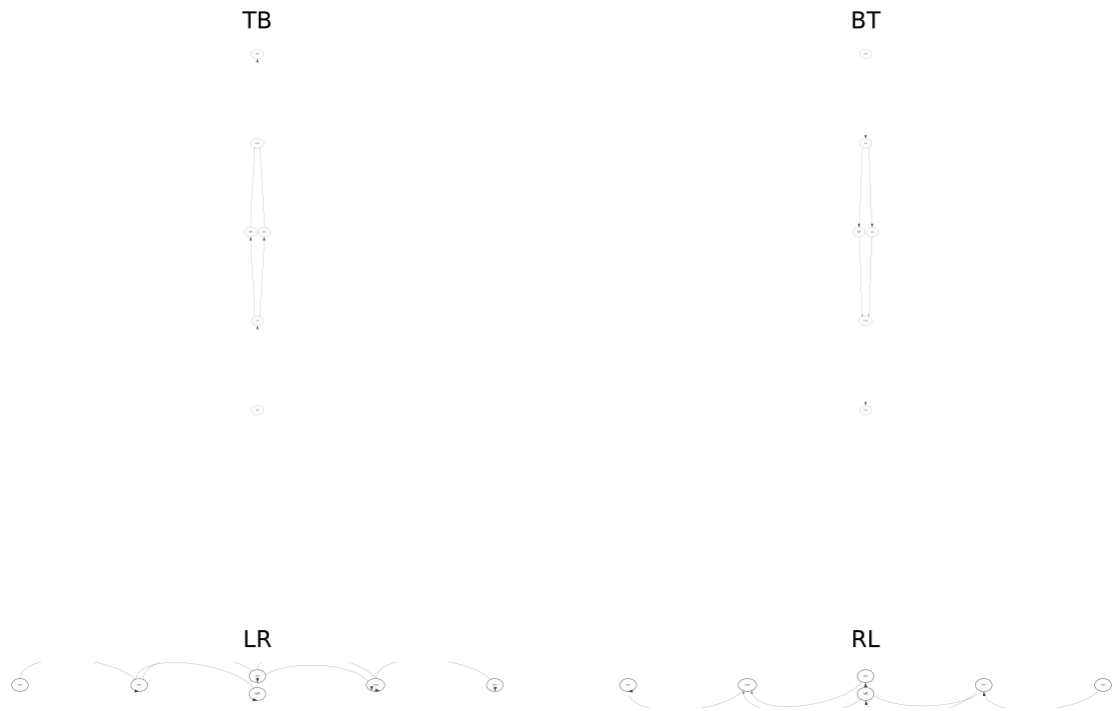


Figure 2: Direction modes change the global flow convention while preserving structure.

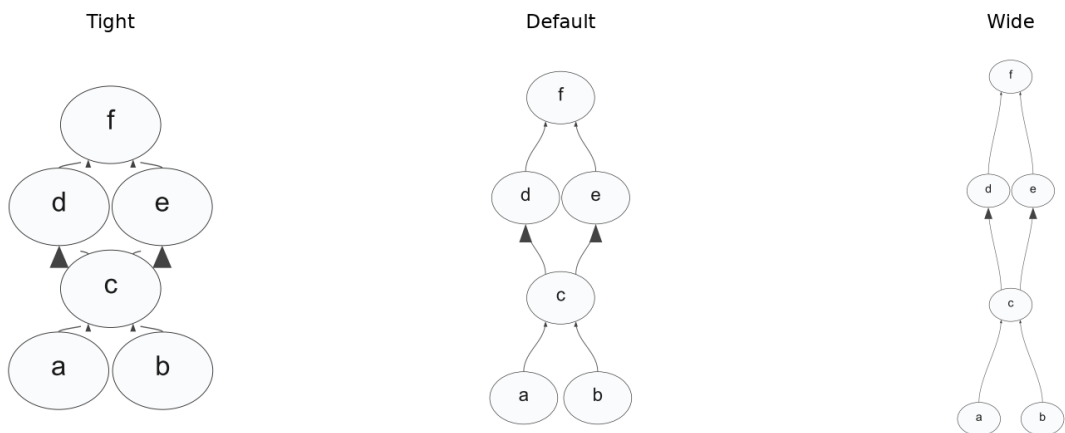


Figure 3: Node and rank separation alter whitespace and breathing room.

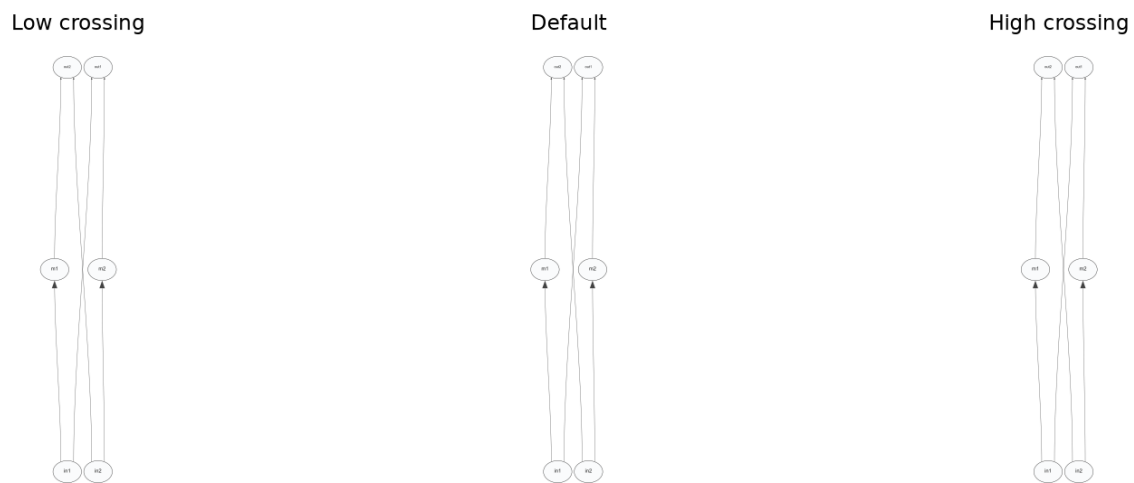


Figure 4: Crossing weight trades local compactness for reduced edge tangles.

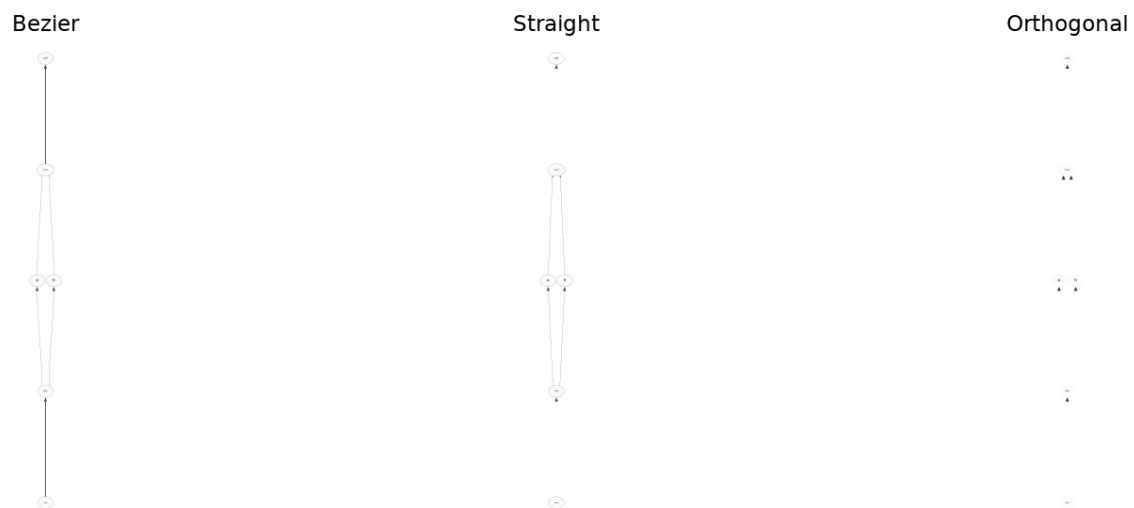
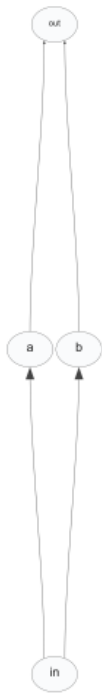


Figure 5: Edge routing style and curvature change the visual character of the same topology.

Default



Pinned + aligned

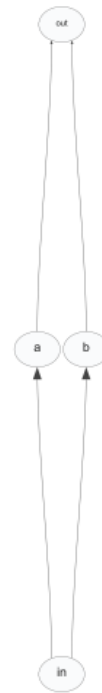


Figure 6: Pins and alignment constraints visibly reshape the same graph.